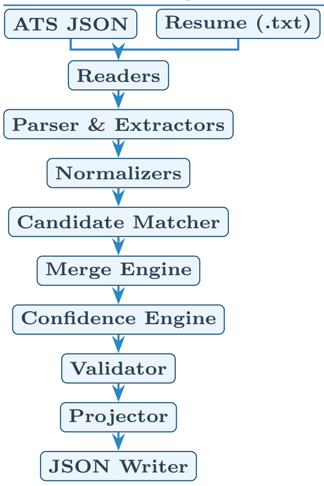
Multi-Source Candidate Data Transformer

Vidish Kumar vidishkumar890@gmail.com github.com/Vidish3442

Eightfold Engineering Intern Assignment — Jul–Dec 2026

Pipeline Design



Each source first becomes a typed model (`ATSCandidate` , `ResumeCandidate`) before matching and merging into the canonical `Candidate` . This isolates source-specific quirks and keeps each stage independently testable.

Canonical Output Schema

```
{candidate_id, full_name, emails[ ],
 phones[ ], location{city, state, country},
 links{ }, headline, years_experience,
 skills[ ], experience[ ], education[ ],
 certifications[ ], provenance[ ],
 overall_confidence}
```

Key constraints: phones in E.164; dates in `YYYY-MM` ; country in ISO 3166-1 alpha-2; all list fields deduplicated; `overall_confidence` $\in [0.0, 0.99]$.

Merge & Conflict Strategy

Matching (`candidate_matcher.py`) runs before merging: exact email → phone digit overlap (≥ 7 digits) → single-candidate fallback.

Scalar fields (name, email, phone): config-driven priority list — first non-empty value in the list wins. Default: `ats > resume` .

List fields (skills, experience, education):

- *Union + dedup* — ATS entries treated as canonical base.
- *Skills*: rapidfuzz `token_sort_ratio` $\geq$ `85` collapses near-duplicates ("py" = "Python").
- *Experience*: matched on *title + company* (score ≥ 80); richer description kept; bullets unioned.
- *Education*: matched on *institution name alone* (score ≥ 85) — avoids misses when ATS stores *"B.Tech CS"* and resume stores *"Bachelor of Technology"* separately.

Every merge decision writes to `ProvenanceTracker` rather than constructing provenance inline, keeping merge logic and audit-trail construction fully decoupled.

Confidence Calculation

$$C_f = \min(0.99,\ W_{\max} + B_{agree} - P_{norm})$$

where:

- $W_{\max}$ = max source weight among contributing sources
- $B_{agree} = 0.05$ if ≥ 2 sources agree, else 0
- $P_{norm} = 0.15$ if normalization failed, else 0

Source weights: ATS = 0.95, Resume = 0.85, Recruiter = 0.70

Overall confidence = weighted average of all field confidences, weighted by field importance (w_f):

$$C_{overall} = \frac{\sum_f C_f \cdot w_f}{\sum_f w_f}$$

Field importance samples:

Field	w_f	Field	w_f
candidate_id	1.0	skills	0.85
full_name	1.0	experience	0.90
emails	1.0	education	0.85
phones	0.9	headline	0.50

Runtime Config Handling

All output shaping lives in `config.yaml` — no code changes needed:

- `fields` — select which fields appear in transformed output
- `rename` — map canonical key to output key (`phone` → `mobile`)
- `include_confidence / include_provenance` — toggle metadata
- `missing_policy` — `null` (keep as JSON null) — `omit` (drop key) — any string (default value)
- `source_priority` — per-field list for scalars; `{merge: union}` for list fields
- `matching.fuzzy_threshold` — rapidfuzz score threshold for candidate identity matching

`ConfigLoader` parses the file; `ConfigValidator` checks shape before the pipeline runs. `MergeEngine` reads `source_priority` ; `Projector` applies the rest.

Edge Cases Handled

- **Scanned PDF** — no extractable text ⇒ warns, continues with ATS-only, lowers confidence
- **Malformed ATS JSON / missing fields** — caught, logs warning, applies `missing_policy`
- **Unparseable phone** — kept as raw string, `normalization_failure` penalty applied
- **Skill aliases** — "py", "k8s", "postgres" mapped via `SKILL_ALIASES` before dedup
- **Split education fields** — institution matched alone; degree/field_of_study reconciled after match
- **Experience spread across blank-line blocks** — forward-scan reassembles title + date + bullets
- **ACHIEVEMENTS bleeding into CERTIFICATIONS** — `ACHIEVEMENTS` added as a known section header
- **Bare year dates** (e.g. "2023") — kept as `YYYY`, not inflated to `YYYY-MM`

Assumptions

- One ATS record + one resume per pipeline run (single-candidate mode)
- Resume is in English with standard section headers (EXPERIENCE, EDUCATION, SKILLS, etc.)
- ATS `candidate_id` is stable and used as the canonical primary key
- Source reliability weights are static (configured, not learned)
- Date strings are parseable by `dateparser` (e.g. "Jan 2023", "2023-01", "Present")